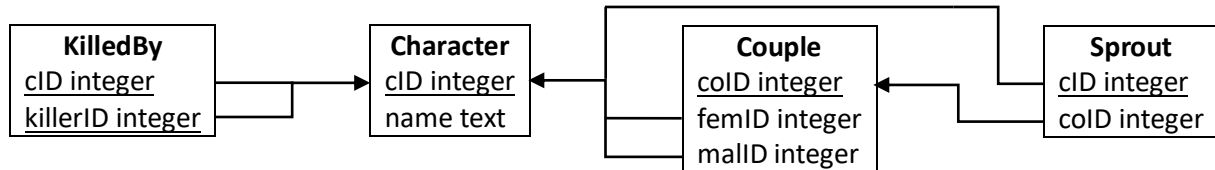


DBMS Klausur 2023 VT Lösung

Aufgabe 1: SQL

(45 Punkte)



a) Alle Character. (cID, ganzerName) (1 Punkt)

```
SELECT cID, name AS ganzerName
FROM Character;
```

b) Alle Character, die nicht getötet wurden. (cID) (1 Punkt)

```
SELECT cID
FROM Character
WHERE cID NOT IN
    (SELECT cID
     FROM KilledBy);
```

c) Die Namen aller Paare. (femName, malName) (2 Punkte)

```
SELECT Female.name AS femName, Male.name AS malName
FROM Character AS Female JOIN Couple ON (Female.cID = Couple.femID)
JOIN Character AS Male ON (Male.cID = Couple.maleID)
```

d) Alle Paare, in denen ein Partner den Namen "Harry Potter" hat. (2 Punkte)

```
SELECT DISTINCT colD
FROM Character AS Female JOIN Couple ON (Female.cID = Couple.femID)
JOIN Character AS Male ON (Male.cID = Couple.maleID)
WHERE Female.name = 'Harry Potter' OR Male.name = 'Harry Potter';
```

e) Alle (direkten) Nachfahren von dem Character mit der ID 42. (cID) (3 Punkte)

```
SELECT cID
FROM Couple NATURAL JOIN Sprout
WHERE femID = 42 OR malID = 42;
```

f) Für alle Character, wie häufig sie einen anderen Character getötet haben (ggf. erscheint die Zahl 0, wenn der Character niemanden getötet hat). (cID, numKills) (3 Punkte)

```
SELECT killerID AS cID, COUNT(KilledBy.cID) AS numKills
FROM Character LEFT JOIN KilledBy ON (Character.cID = KilledBy.killerID)
GROUP BY killerID;
```

g) Alle Character, die mit mindestens 3 unterschiedlichen Personen ein Paar gebildet haben. (cID) (3 Punkte)

```
SELECT femID AS cID
FROM Couple
GROUP BY femID
HAVING COUNT(DISTINCT malID) >= 3
UNION
SELECT malID AS cID
FROM Couple
GROUP BY malID
HAVING COUNT(DISTINCT femID) >= 3;
```

h) Alle Character, die von ihrem Partner getötet wurden. (cID) (3 Punkte)

```
SELECT cID
FROM Character NATURAL JOIN KilledBy
JOIN Couple ON (KilledBy.cID = Couple.femID OR KilledBy.cID = Couple.malID)
WHERE KilledBy.killerID = Couple.malID OR KilledBy.killerID = Couple.femID;
```

i) Den kinderreichsten Character. (cID) (4 Punkte)

```
SELECT cID
FROM Character JOIN Couple ON (Character.cID = Couple.femID OR Character.cID = Couple.malID)
WHERE colD IN
    (SELECT colD
     FROM Sprout
     GROUP BY colD
     HAVING COUNT(cID) >= ALL
        (SELECT COUNT(cID)
         FROM Sprout
         GROUP BY colD));
```

j) Alle Nachkommen, die beide Elternteile getötete haben. (cID) (4 Punkte)

```
SELECT Sprout.cID
FROM Sprout NATURAL JOIN Couple JOIN KilledBy ON (Sprout.cID = KilledBy.killerID)
WHERE KilledBy.cID = Couple.femID AND KilledBy.cID = Couple.malID;
```

k) Das Paar, das zusammen die meisten Character getötet hat. (4 Punkte)

```
SELECT colD
FROM Couple
JOIN KilledBy AS femKills ON (Couple.femID = femKills.killerID)
JOIN KilledBy AS malKills ON (Couple.malID = malKills.killerID)
GROUP BY colD
HAVING COUNT(DISTINCT femKills.cID) + COUNT(DISTINCT malKills.cID) >= ALL
    (SELECT COUNT(DISTINCT femKills.cID) + COUNT(DISTINCT malKills.cID)
     FROM Couple
     JOIN KilledBy AS femKills ON (Couple.femID = femKills.killerID)
     JOIN KilledBy AS malKills ON (Couple.malID = malKills.killerID)
     GROUP BY colD);
```

l) Jeden Character und dessen Folgecharacter (also der, der lt. cID als nächstes in der Tabelle Character erscheint – es muss nicht durchgehend nummeriert sein. (cID, folgeChar) (4 Punkte)

```
SELECT cID, LEAD(cID) OVER (ORDER BY cID) AS folgeChar
FROM character;
```

m) Alle Character, die sowohl ganze Paare als auch alle deren Nachkommen getötet haben. (cID)

WITH Victims(cID) **AS** (5 Punkte)

(**SELECT** femID, colD

FROM Couple

UNION

SELECT malID, colD

FROM Couple

UNION

SELECT clD, colD

FROM Sprout)

SELECT killerID

FROM KilledBy

GROUP BY killerID

HAVING ARRAY_AGG(cID) @> **ANY**

(**SELECT ARRAY_AGG**(cID)

FROM Victims

GROUP BY colD);

n) Stellen Sie sicher, dass in der Couple-Tabelle jeder Character maximal 3x vorhanden ist. (6 Punkte)

CREATE FUNCTION checkCouple() **RETURNS TRIGGER AS**

\$\$

BEGIN

IF EXISTS

(**SELECT COUNT**(femID) **FROM** Couple **GROUP BY** femID **HAVING COUNT**(femID) > 3)

UNION

(**SELECT COUNT**(malID) **FROM** Couple **GROUP BY** malID **HAVING COUNT**(malID) > 3)

THEN RAISE EXCEPTION 'Jeder Character darf nur maximal 3x verheiratet (gewesen) sein.';

END IF;

RETURN NULL;

END;

\$\$ **LANGUAGE** plpgsql;

CREATE TRIGGER checkCouple

AFTER INSERT OR UPDATE ON Couple

FOR EACH ROW

EXECUTE FUNCTION checkCouple();

Aufgabe 2: Etc

(17 Punkte)

a) Welche Einschränkungen existieren in Postgres hinsichtlich der CHECK-Klausel? (2 Punkte)

In einer CHECK Überprüfung können Unterabfragen und Aggregatfunktionen nicht verwendet werden.

b) Welche Eigenschaften gelten für ein Attribut, wenn es als Primary Key definiert wurde? (1 Punkt)

Das Attribut darf nicht doppelt vorkommen und es darf nicht den Wert NULL annehmen.

c) Gegeben sei die Tabelle R. Was geben die folgenden Statements zurück? Schreiben Sie die Zahl direkt hinter das Statement. (5 Punkte)

1. **SELECT COUNT(DISTINCT x) FROM R;** --3 Datensätze
2. **SELECT COUNT(*) FROM R;** --6 Datensätze
3. **SELECT COUNT(x) FROM R;** --4 Datensätze
4. **SELECT COUNT(*) FROM R r1 JOIN R r2 USING(x);** --6 Datensätze
5. **SELECT COUNT(*) FROM R r1 NATURAL JOIN R r2;** --6 Datensätze

x
1
2
2
3
NULL
NULL

d) Vervollständigen Sie die Wahrheitstabelle des OR Operators.

OR	True (T)	False (F)	Null (U)
True (T)	T	T	T
False (F)	T	F	U
Null (U)	T	U	U

(2.5 Punkte)

e) Welche Probleme existieren beim Einfügen in Tabellen, die sich gegenseitig durch Fremdschlüssel referenzieren? Wie kann das Problem gelöst werden? (2.5 Punkte)

Bei zwei Tabellen die sich gegenseitig durch Fremdschlüssel referenzieren können keine Datensätze hinzugefügt werden, da sonst immer ein Fremdschlüsselconstraint verletzt werden würde. Umgehen kann man dieses Problem, indem man die Überprüfung des Constraints temporär ans Ende einer Transaktion verschiebt und zwischenzeitlich die neuen Datensätze hinzufügt.

f) Welche drei Möglichkeiten existieren, um in SQL die referenzielle Integrität (m.H. der Definition von Foreign Keys) beim Update oder Delete sicherzustellen? Nennen Sie die drei Möglichkeiten und erläutern Sie diese kurz (ein Satz). (3 Punkte)

Mögliche Reaktionen in einer Fremdschlüsseltabelle:

1. **CASCADE**: Beim Update / Delete soll sich die Aktion fortpflanzen
2. **RESTRICT**: Beim Update / Delete soll die Aktion abgelehnt werden
3. **SET DEFAULT** bzw. **SET NULL**: Beim Delete soll der Standardwert bzw. NULL eingefügt werden

g) Warum ist folgendes Statement für die Tabellen X(a) und Y(b) falsch? (1 Punkt)

SELECT * FROM X NATURAL JOIN Y WHERE COUNT(a) > 10;

Die Anfrage ist syntaktisch fehlerhaft, da einerseits eine Aggregatfunktion in der WHERE-Klausel statt in der HAVING-Klausel verwendet wird, und andererseits, weil man im SELECT die Attribute a und b ausgeben möchte während man eine Aggregatfunktion verwendet, ohne auf sie gruppiert zu haben.

Aufgabe 3: Normalisierung

(7 Punkte)

Gegeben sind für die Relation $R(a, b, c, d, e, f, g, h, i, j)$ die funktionalen Abhängigkeiten

1. $\{a\} \rightarrow \{f\}$
2. $\{b, c\} \rightarrow \{g, h\}$
3. $\{d, e\} \rightarrow \{i, j\}$
4. $\{g\} \rightarrow \{h\}$
5. $\{e\} \rightarrow \{j\}$

a) Nennen Sie die Eigenschaften von Schlüsselkandidaten.

(1 Punkt)

Schlüsselkandidaten determinieren jedes Tupel eindeutig und sind minimal. Minimal bedeutet, dass kein Attribut aus der Menge entfernt werden kann, ohne, dass die erste Eigenschaft verloren geht.

b) Nennen Sie die Schlüsselkandidaten der Relation.

(1 Punkt)

Der einzige Schlüsselkandidat von R ist die Menge $\{a, b, c, d, e\}$.

c) Nennen Sie die funktionalen Abhängigkeiten, die ggf. im Widerspruch zur 2ten und dritten Normalform stehen.

(2 Punkte)

Widerspruch 2te NF:

1. $\{a\} \rightarrow \{f\}$
2. $\{b, c\} \rightarrow \{g, h\}$
3. $\{d, e\} \rightarrow \{i, j\}$
5. $\{e\} \rightarrow \{j\}$

Widerspruch 3te NF:

4. $\{g\} \rightarrow \{h\}$

d) Überführen Sie die Relation in die dritte Normalform und kennzeichnen Sie Primär- und Fremdschlüssel.

(3 Punkte)

Unnormalisiert:

$R(a, b, c, d, e, f, g, h, i, j)$

Normalisiert in die dritte Normalform:

$R1(\underline{a}, b, c, d, \underline{e})$, mit Fremdschlüssel $\{a\}$ zu $R2$, $\{b, c\}$ zu $R3$, $\{d, e\}$ zu $R5$ und $\{e\}$ zu $R6$

$R2(\underline{a}, f)$

$R3(\underline{b}, c, g)$, mit Fremdschlüssel $\{g\}$ zu $R4$

$R4(\underline{g}, h)$

$R5(\underline{d}, \underline{e}, i)$, mit Fremdschlüssel $\{e\}$ zu $R6$

$R6(\underline{e}, j)$

Aufgabe 4: Optimierung / Relationale Algebra

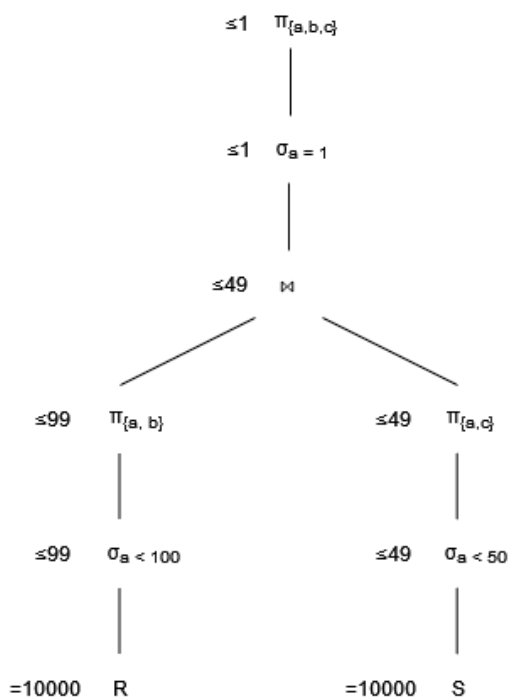
(8 Punkte)

Für gegebene Relationen $R(a, b)$ und $S(a, c)$, welche beide 10000 Datensätze enthalten und welche beide dem Primärschlüssel a haben, ist das unten angegebene Statement gegeben (es gilt, dass a vom Typ integer ist, und dass $a > 0$ ist).

SELECT *
FROM (**SELECT** * **FROM** R **WHERE** $a < 100$) **NATURAL JOIN**
 (**SELECT** * **FROM** S **WHERE** $a < 50$)
WHERE $a = 1$;

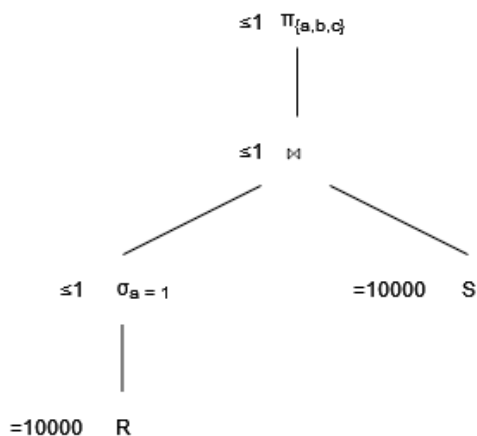
a) Zeichnen Sie für das Statement den Operatorbaum und kennzeichnen Sie die Knoten mit der erwarteten Anzahl an Datensätzen. (2 Punkte)

Operatorbaum:



b) Zeichnen Sie für ein äquivalentes Statement einen Operatorbaum, dessen Summe an inneren Knoten geringer ist (Sie müssen nicht zeigen, dass das Statement äquivalent ist). (3 Punkte)

Operatorbaum:



c) Geben Sie die formale Definition des kartesischen Produkts in der relationalen Algebra an. (1 Punkt)

Definition des kartesischen Produkts:

$$Rel_m \times Rel_t := \{ \langle w_1, \dots, w_m, w_{m+1}, \dots, w_{m+t} \rangle \mid \\ \exists t_m \in Rel_m (t_m(attr_1, \dots, attr_m) = \langle w_1, \dots, w_m \rangle) \wedge \\ \exists t_n \in Rel_t (t_n(attr_{m+1}, \dots, attr_{m+t}) = \langle w_{m+1}, \dots, w_{m+t} \rangle) \}$$

d) Geben Sie die formale Definition der Projektion in der relationalen Algebra an. (1 Punkt)

Definition der Projektion:

$$\pi_{\{attr_1, \dots, attr_m\}}(Rel) := \{ \langle w_1, \dots, w_m \rangle \mid \{attr_1, \dots, attr_m\} \subseteq \{attr_1, \dots, attr_n\}^{Rel} \\ \wedge \exists t \in Rel (t = \langle w_1, \dots, w_m, \dots, w_n \rangle) \}$$

e) Geben Sie die formale Definition der Selektion in der relationalen Algebra an. (1 Punkt)

Definition der Selektion:

$$\sigma_{Bedingung}(Rel) := \{ t \mid t \in Rel \wedge Bedingung(t) = true \}$$

Aufgabe 5: Synchronisation und Isolationsstufen

(8 Punkte)

- a) Zeigen Sie anhand eines selbstgewählten Beispiels, dass das strikte Zweiphasensperrprotokoll nicht deadlockfrei ist (führen Sie dazu das Protokoll für das Beispiel aus – das Nennen eines Schedules reicht nicht als Erklärung).

(3 Punkte)

Schedule: R1(x) R2(x) W1(x) W2(x)

Striktes Zweiphasensperrprotokoll:

Aktion	x	Warteschlange
R1(x)	S1 R1	
R2(x)	S2 R2	
W1(x)	⌘	W1(x)
W2(x)	⌘	W1(x) W2(x)

Die Transaktionen T₁ und T₂ warten wechselseitig auf die Freigabe von Datenobjekt x (Deadlock).

- b) Gegeben seien die unten angegebenen parallel ablaufenden Sitzungen zweier Benutzer in Postgres für eine Tabelle X(a INTEGER, b INTEGER) mit einem Datensatz (1,1).

Schreiben Sie neben die Updates bzw. Commits, falls eine Transaktion wartet, wann eine Aktion dazu führt, dass die andere Transaktion fortgeführt wird, bzw. wann eine Transaktion abbricht. Geben Sie abschließend an, welche Werte in der Datenbank stehen. (5 Punkte)

Szenario 1:

Nr	Benutzer 1 (Transaktion 1)	Benutzer 2 (Transaktion 2)
1	BEGIN;	
2	SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;	
3		BEGIN;
4		SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
5	UPDATE X SET b = 4 WHERE a = 1;	
6		UPDATE X SET b = 5 WHERE a = 1;
7	COMMIT;	
8		COMMIT;

Zwischenschritte und Endresultat:

Nr	REPEATABLE READ
5	T1 schreibt
6	T2 kann nicht schreiben, wartet
7	T1 beendet, T2 bricht ab
8	T2 ROLLBACK
⇒	(1, 4)

Szenario 2 (gestartet wird bei a = 1, b = 1):

Nr	Benutzer 1 (Transaktion 1)	Benutzer 2 (Transaktion 2)
1	BEGIN;	
2	SET TRANSACTION ISOLATION LEVEL READ COMMITTED;	
3		BEGIN;
4		SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
5	UPDATE X SET b = 2 WHERE a = 1;	
6		UPDATE X SET b = 3 WHERE a = 1;
7	COMMIT;	
8		COMMIT;

Zwischenschritte und Endresultat:

Nr	READ COMMITTED
5	T1 schreibt
6	T2 kann nicht schreiben, wartet
7	T1 beendet, T2 setzt fort
8	T2 beendet
⇒	(1, 3)

Aufgabe 6: Scheduling

(5 Punkte)

Gegeben sei der folgende Schedule:

R1(x) R2(y) R3(z) W2(x) W3(y) W1(x) W1(z) W3(z)

Welcher (um die Sperren erweiterte) Ergebnisschedule würde mit Preclaiming für die oben genannte Schedule erzeugt?

Zweiphasensperrprotokoll mit Preclaiming:

Aktion	x	y	z	Warteschlange
R1(x)	X1 R1		X1	
R2(y)	↯			R2(y)
R3(z)			↯	R2(y) R3(z)
W2(x)	↯			R2(y) R3(z) W2(x)
W3(y)			↯	R2(y) R3(z) W2(x) W3(y)
W1(x)	W1 U1			R2(y) R3(z) W2(x) W3(y)
→ R2(y)	X2	S2 R2 U2		R3(z) W2(x) W3(y)
→ W2(x)	W2 U2			R3(z) W3(y)
W1(z)			W1 U1	R3(z) W3(y)
→ R3(z)		X3	X3 R3	W3(y)
→ W3(y)		W3 U3		
W3(z)			W3 U3	

Hilfsnotizen fürs Preclaiming: $T_1: X(x), X(z)$ $T_2: X(x), S(y)$ $T_3: X(y), X(z)$